

**UNIVERSIDAD PERUANA DE CIENCIAS APLICADAS**

FACULTAD DE INGENIERÍA

CARRERA DE CIENCIAS DE LA COMPUTACIÓN

**Curso:** Tópicos en Ciencias de la Computación

# PROYECTO MURDOKU

## TRABAJO PARCIAL

**Profesor:** Willy Gustavo Ugarte Rojas

**Estudiantes:**

Canchis Fernández, Mauricio Andrés - U202318106

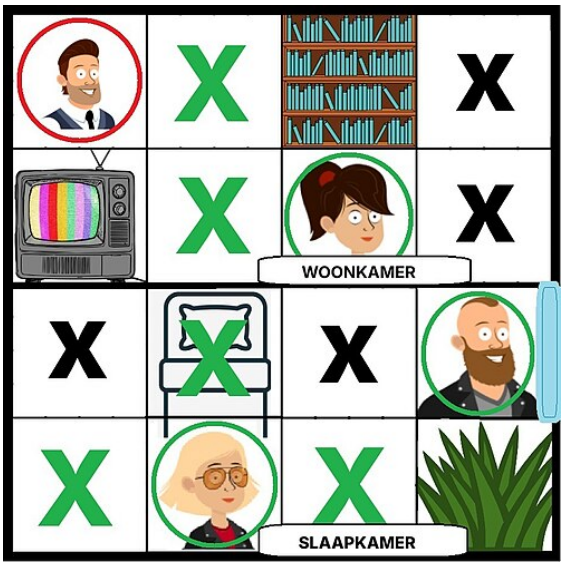
Colfer Mendoza, Carlos Alejandro - U20241B820

Mucha Álvarez, Diego Fabrizio - U202317069

**Ciclo: 2026-02**

1. **Murdoku**

   Murdoku está basado en el popular juego Sudoku, creado recientemente por Manuel Garand. No obstante, en lugar de barajar números y rellenar todas las casillas vacías, el jugador se enfrasca en una didáctica de juegos de misterio. Así, cada casilla representa la ubicación de una localización, y el jugador permuta y tantea la ubicación en el momento del crimen de común denominador de este juego: un asesinato. El objetivo es descubrir en dónde se hallaba finalmente el perpetrador del homicidio, reconstruyendo una labor detectivesca de forma peculiar y entretenida (Alegre Herrero, 2026).




Ejemplo de una malla tipo Sudoku del juego Murdoku

   Su uso se ha popularizado, y ahora se extiende a la temática del Trabajo Parcial y el presente informe.

2. **Objetivos**

   Construir una solución al juego de Murdoku valiéndonos de aprendizajes y herramientas vistas a lo largo del curso: Constraint Programming (programación de restricciones para problemas de computación, OR-TOOLS) y procesar un ejemplo con visión computacional para la resolución del problema. Es una amalgama del paradigma de programación por restricciones y el procesamiento de imágenes. La validación se materializará mediante pruebas con imágenes de Murdoku no previstas en el entrenamiento del modelo de visión computacional.

3. **Explicación del pipeline de Visión Computacional**

   La herramienta de visión computacional empleada para este proyecto es Tesseract OCR (Reconocimiento Óptico de Caracteres por sus siglas en inglés), motor de código abierto y con librerías disponibles para el lenguaje de programación Python, el requerido y más óptimo para este trabajo. Su API es aplicable desde la línea de comandos, y está disponible en una variedad de idiomas. Está disponible bajo licencia Apache 2.0 (Tesseract OCR Project, s.f.).

   En el proyecto, empleamos dos archivos pertinentes: ocr.py y board_reacher.py. En ocr.py, se recorta el área lateral o superior de la imagen donde se ubiquen las palabras escritas. En la función preprocess_for_ocr(), se maximizan el contraste escalando a grises y la posterior binarización Otsu (separación de objeto del fondo). Posteriormente, se llama al modelo Tesseract OCR y extrae palabras individuales con un umbral de confianza mínimo (50). La función _spit_clues_by_person() busca el nombre de los sospechosos con expresiones regulares (Regex) y segmenta el texto necesario para la identificación. Con _parse_person_clues() se aplica un intenso Regex para capturar pistas textuales y someterlas a las pistas o expresiones ligadas a objetos o secciones de la cuadrícula Murdoku (si una palabra hace referencia a la posición de ubicación de una cosa, como ‘ ’top’ ’, ‘ ’next’ ’, escritas como pistas).

   Con board_reacher.py, se procesa el plano visual de la grilla de Murdoku mediante la herramienta OpenCV, también de visión computacional. Detecta y aísla el tablero, buscando el contorno de mayor área y de silueta rectangular (cv2.boundingRect). Luego, se divide la región del tablero en n x n (6 x 6 por defecto), y se calculan las coordenadas de cada celda. Se mapean los atributos del tablero,e establece una relación de nombre de archivos con los colores sujetos, y como colofón, la función analyse_murdoku_json() retorna la matriz completa del juego en formato JSON, indicando el ID del cuarto y objeto presente en las cuadrículas.

4. **Modelo Matemático formal de Constraint Programming**

   El modelo matemático formal de Constraint Programming utiliza la librería OR-Tools (CP-SAT) para satisfacer una serie de restricciones (CSP), directamente relacionado con el enfoque de las investigaciones de la Inteligencia Artificial (Brailsford et al., 1999). El programa traduce las reglas del juego a un modelo matemático, y el solver alcanza la respuesta unaria en cuestión de segundos.

   Las variables del modelo matemático de CP se establecen en función de las personas no tangibles de las mallas: fila (fp), columna (cp), habitación (hp) y si comparte el cuatro con la víctima (bp). Además, se garantiza que solo se ubique una persona en una casilla (fila tal,

columna tal) y no más de una (AddAllDifferent). Se crearon reglas (escritos, texto) como ‘ ’mismo_cuarto’ ’, ‘ ’solo_con’ ’ o ‘ ’al_norte_de’ ’. Estas hacen de ecuaciones lineales simples entre las posiciones de los personajes en el tablero.

Adicionalmente, se valida que solo un sospechoso se encuentre con la víctima fatal, soloa una persona. Finalmente, se subdivide el tablero en secciones de una vivienda: cocina, sala baño, jardín, dormitorio; y se asegura la unicidad de la respuesta, de forma que solo existe una solución válida.

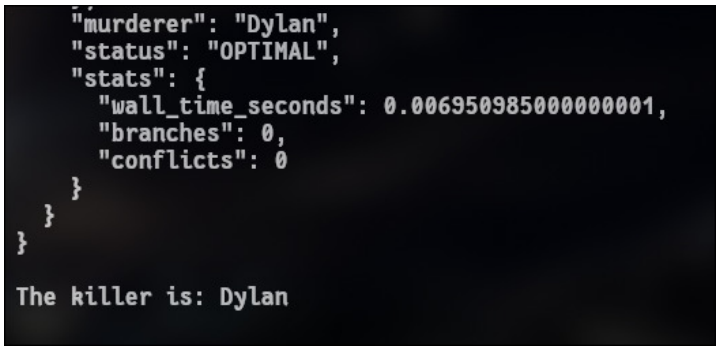
```python
##el asesino es el único que comparte cuarto con la víctima
con_victima = {}
for p in personas:
  if p != v:
    b = model.NewBoolVar('con_victima_' + p)
    #b => mismo cuarto
    model.Add(cuarto[p] == cuarto[v]).OnlyEnforceIf(b)
    #not b => distinto cuarto
    model.Add(cuarto[p] != cuarto[v]).OnlyEnforceIf(b.Not())
    con_victima[p] = b
model.AddExactlyOne(con_victima.values())

return model, {'fila': fila, 'col': col, 'cuarto': cuarto, 'con_victima': con_victima}
```

```
Estado: OPTIMAL | tiempo: 0.0277 s | ramas: 0 | conflictos: 0
Ana    -> fila 0, columna 1, Sala
Bruno  -> fila 1, columna 4, Cocina
Carla  -> fila 2, columna 0, Dormitorio
Diego  -> fila 3, columna 3, Baño
Elena  -> fila 4, columna 5, Jardín
Vera   -> fila 5, columna 2, Dormitorio
Asesino: Carla
```

5. **Análisis de complejidad y tiempos de respuesta del Solver**

   Los tiempos de respuesta del solver son veloces a pesar de su notoria complejidad robusta en su arquitectura: desde la visión computacional hasta los requisitos CSP tallados en código en Python. La respuesta se arroja en formato JSON, o parte de ella, y gracias a esta tenemos evidencia de un ejemplo apra notificar y esclarecer la velocidad de respuesta del solver construido con minucioso cuidado:




Podemos apreciar que el tiempo de respuesta de 0.00695… segundos, menos de un segundo, lo cual prueba su óptimo funcionamiento y eficiente rapidez de ejecución.

6. **Bibliografía**

Alegre Herrero, M. (1 de junio de 2026). *¿Qué es el 'Murdoku'? El juego que mezcla Sudoku y Cluedo, arrasa en TikTok y ya es el libro más vendido*. El Periódico Mediterráneo. https://www.elperiodicomediterraneo.com/sociedad/2026/06/01/murdoku-pasatiempo-detective-acertijos-130804892.html

Brailsford, S. C., Potts, C. N., & Smith, B. M. (1999). Constraint satisfaction problems: Algorithms and applications. *European Journal of Operational Research, 119*(3), 557–581. https://doi.org/10.1016/S0377-2217(98)00364-6

Tesseract OCR Project. (s.f.). *Tesseract documentation*. Recuperado el 5 de octubre de 2026: https://tesseract--ocr-github-io.translate.goog/tessdoc/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=sge